# Coordinated Scheduling and Process-Mode Selection for Dual-Source Rotating-Chamber Cluster Tools

***Abstract*—Finite-batch scheduling of rotating-chamber cluster tools requires coordination of product supply, reusable dummy wafers, and periodic cleaning. This paper develops a mixed-integer linear programming model for dual-source mixed flows under full-occupancy, residency, and shared-resource constraints. A makespan-first second stage refines waiting and processing cadence within the first-stage bound. Structure-preserving initial schedules and role-aware hierarchical cut selection support time-limited solution. Computational mode comparisons use prescribed wafer-pair layouts and equal cumulative nominal processing times. The preferred mode changes with lot size and cleaning-induced chain segmentation; for 16 wafers without activated cleaning, chained processing reduces product-return makespan by 4.87% relative to parallel processing. On the frozen algorithm benchmark, equal-budget two-stage optimization reduces total route wait by 51.91%, while precomputed initial schedules increase incumbent availability from 51.67% to 100% during the subsequent solve. The learned cut selector and its hierarchical baseline have similar observed performance without a statistically established difference. These results support conditional process-mode comparison and coordinated event scheduling; physical recipe equivalence and production deployment are not established by the computational experiments.**

*Note to Practitioners*—Engineers operating semiconductor cluster tools must coordinate production wafers with reusable dummy wafers that occupy otherwise empty positions and support chamber cleaning. A faster chamber cycle does not necessarily finish a finite lot sooner: dummy returns, robot access, and the point at which cleaning interrupts a processing chain can change the preferred operating mode. The proposed scheduler takes qualified recipes, equipment capacities, action durations, a wafer-pair layout, and available dummy inventory as inputs. It returns timed handling, processing, and cleaning actions, and can refine waiting after obtaining a feasible completion-time bound. Its intended use is offline evaluation of operating alternatives and preparation of candidate schedules. Application requires checking the input timings and occupancy rules against the actual tool and validating the resulting actions with the equipment control system. The reported tests use deterministic computational instances; they do not demonstrate film-quality equivalence, disturbance recovery, or production-line deployment. Initial-schedule construction also consumes time outside the reported main-solver budget. Users with a strict decision deadline should include this preparation cost and retain a validated fallback schedule.

***Index Terms*—semiconductor manufacturing system , cluster tool , dual-source material flow , operational decision , mixed-integer linear programming , learning-enhanced exact optimization**

## I. Introduction

Cluster tools coordinate processing modules, robots and load locks in semiconductor fabrication. Residency limits, shared handling resources and cleaning couple wafer routes and production output [1]. Multi-space modules and multi-finger robots add parallel processing and paired handling opportunities [2], [3]. This paper studies finite batches in plasma-enhanced chemical vapor deposition (PECVD) equipment with two four-position rotating chambers.

Products enter from atmospheric load ports; reusable dummy wafers enter from vacuum storage. Under the prescribed full-occupancy recipe, dummies fill unused positions and occupy chambers during cleaning. Their return times constrain subsequent product processing. We call these separate supplies dual-source operation, and their interleaved use of robots and chambers mixed flow. The scheduler must jointly determine product and dummy movements, robot service order, and the timing of complete processing and cleaning chains.

Two qualified processing modes are compared conditionally: 4×1 processes four wafers once, whereas 2×2 alternates paired exchanges so that every product receives two consecutive exposures. Equal cumulative nominal process time allows a scheduling comparison; it does not establish physical recipe equivalence. Cycle counts and chain initialization/termination can move cleaning inside or after the product-processing horizon. Consequently, the mode with the shortest individual cycle need not complete the lot first.

Prior work develops throughput analysis, residency-constrained linear programming and timed Petri-net models [4]–[7]. Extensions include revisiting routes, multiple wafer types, selectable modules and joint release/handling decisions [8]–[11]. Cleaning-aware models coordinate task sequences, wafer delay and cleaning triggers [12]–[15]. Recent T-ASE work addresses failure-related scheduling and noncyclic reinforcement-learning control [16]–[18]. These studies motivate coordinated automation, but their schedules cannot be transferred unchanged to the present dual-source routes and rotating-chamber occupancy rules.

Learning can also assist mathematical optimization through cut ranking or selection [19]–[21]. The hierarchical sequence model (HEM) chooses the number and ordered subset of cuts [22]. Here, mathematical variable roles augment that representation while the MILP retains responsibility for process feasibility. Numerical comparisons include HEM, configured SCIP, adaptive cut scoring and component ablations on identical models.

The contributions are: 1) a joint finite-batch model of product/dummy logistics, full occupancy, periodic cleaning and shared-resource timing under prescribed pair layouts; 2) makespan-first timing refinement and bound-based identification of how cleaning-induced chain segmentation changes mode preference; and 3) a solver framework combining structure-preserving binary-skeleton (SPBS) initialization and role-aware cut selection, with separate evidence for initializa-

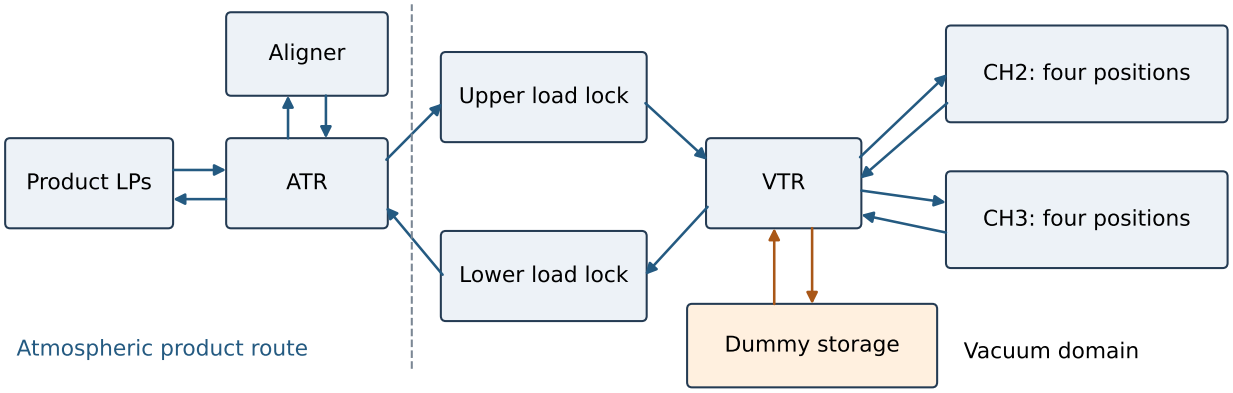



Fig. 1. Dual-source rotating-chamber tool and product/dummy routes.

tion, timing quality and search performance. The experiments support SPBS and timing refinement benefits; they do not establish statistically significant superiority of the complete learned selector over HEM.

## II. Problem description and two-stage scheduling model

### A. Equipment configuration and scheduling task

Given the product lot, process modes, wafer-pair layout, and initial equipment state, scheduling determines the start and finish times of loading, alignment, handling, rotation, processing, cleaning, and return actions, together with their order on shared resources. Products depart from LPs and return after processing. Dummies leave vacuum storage, perform filling or cleaning tasks, and return. Robot service, chamber occupancy, and dummy inventory couple the routes.

Inputs comprise action durations, resource capacities, residency limits, cleaning intervals, and dummy counts. Stage one minimizes the final product return time. Stage two improves waiting and cadence under the resulting makespan bound. Each feasible schedule is represented by an action sequence specifying wafers, resources, and start and completion times for performance evaluation and checks of precedence and resource conflicts.

Action durations are deterministic within a solve. No buffers are assumed beyond LPs, aligners, load locks, rotating chambers, and vacuum storage, so each wait corresponds to a material location and resource-occupancy interval.

### B. PW-pair modeling for the two process modes

To represent wafer-level residency and paired handling together, shared transfer and chamber occupancy are modeled at the pair level, while alignment and route timing retain wafer-level events.

A wafer pair (PW pair) is a physical two-wafer transport and chamber-occupancy unit. Its members may be product+product (product PW pair), dummy+dummy (dummy PW pair), or product+dummy (mixed PW pair). Product wafers are preferentially paired by mode and index; an odd tail product may be paired with one dummy, while full-occupancy filling and cleaning may use dummy PW pairs. The sets $\mathcal{P}^F$ and $\mathcal{P}^L$ index product-containing PW pairs, including product PW pairs and any required mixed PW pairs. Alignment and residency remain wafer-level operations; the two members share a time window only on a selected two-hand transfer or the same chamber side. The prescribed layout specifies product-containing PW pairing and compatible chamber positions. The MILP optimizes the remaining activation and assignment decisions, physical dummy identities, paired transfers, resource order, and event times within this layout. Legal cleaning segments are generated from the cleaning interval and complete recipe boundaries, after which the MILP activates the cleaning tasks and schedules their times.

A product follows LP–ATR–AL–ATR–upper load lock–VTR–rotary chamber–VTR–lower load lock–ATR–LP: the ATR first places the wafer on the aligner, then retrieves it for transfer into the vacuum domain. A dummy travels from vacuum storage through the VTR to the chamber and returns through the VTR after use. A regular all-product $4 \times 1$ batch contains two product PW pairs, hence four product wafers, and processes all four positions synchronously once before last-in–first-out unloading; only the last incomplete batch may use a dummy or mixed PW pair to fill all positions. In $2 \times 2$, every product-containing PW pair occupies two adjacent process cycles so that each of its product members is processed twice. For three product PW pairs, a dummy PW pair D is loaded first, the chamber rotates, and P1 is loaded before D+P1 undergoes chamber processing. Chamber processing then alternates with "chamber rotation + chain slide", changing the occupancy successively to P1+P2, P2+P3, and P3+D; P1, P2, and P3 are each processed twice before all wafers are unloaded. Figures 2 and 3 show the event sequences.

The model uses chambers $c \in C$, product wafers $w \in \mathcal{W}$, product-containing pairs $p \in \mathcal{P}^F \cup \mathcal{P}^L$, batch positions $b \in \mathcal{B}_c$, chain positions $k \in \mathcal{K}_c$, cycles $q \in \mathcal{Q}_c$, and cleaning segments $e \in \mathcal{E}_c$. Physical operations $o \in O$ have start/end times $s_o, e_o$, duration $d_o$, and binary activation $b_o$. Binary $u$ denotes batch/position activation, $f$ dummy filling, $t$ or $h$ a tail, $g$ dummy assignment, $h_{o\ell}$ slot use, and $y$ resource order. Bars denote prescribed input data. $M$ is a valid horizon bound. The supplementary material provides the complete notation catalogue and expanded explanations.

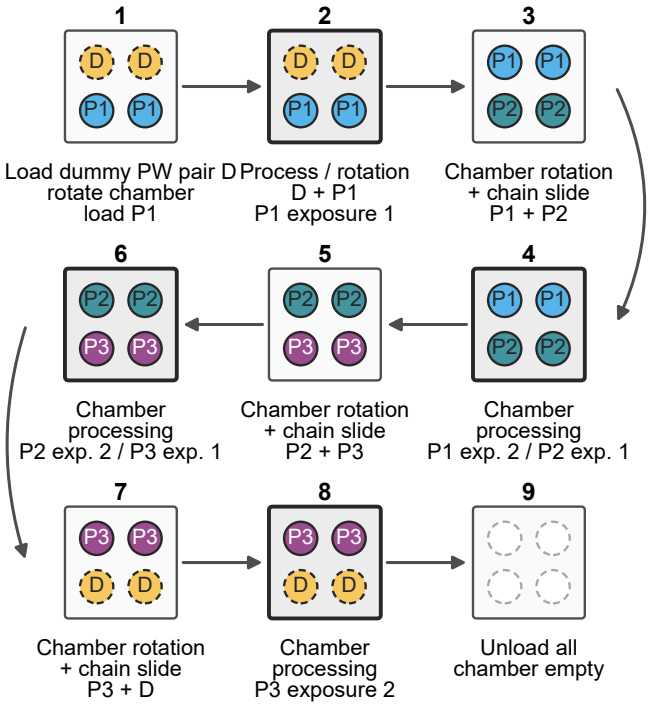



Fig. 2. $2 \times 2$ chained processing. A PW pair may be P+P, D+D, or P+D; here each P$i$ is a P+P product PW pair and D is a D+D dummy PW pair. Step 1 loads D, rotates the chamber, and loads P1; Step 2 performs chamber processing on D+P1; Step 3 applies chamber rotation + chain slide to obtain P1+P2. Repeating "chamber processing–chamber rotation + chain slide" makes P1, P2, and P3 participate in two adjacent process cycles, so every product wafer is processed twice before all wafers are unloaded.

### C. Source interfaces, operation activation, and time relations

The fixed incidence parameter satisfies

$$\sum_{s \in \mathcal{S}} \chi_{is} = 1, \qquad i \in \mathcal{I}. \tag{1}$$

If $\mathcal{A}^{\text{int}}$ is the internal network and $\mathcal{A}_s$ contains release and return arcs for source $s$, then

$$\mathcal{A} = \mathcal{A}^{\text{int}} \cup \left( \bigcup_{s \in \mathcal{S}} \mathcal{A}_s \right). \tag{2}$$

Canonical pair positions satisfy

$$\begin{aligned} x^F_{pcbl} &= \bar{x}^F_{pcbl}, \quad \sum_{c,b,l} x^F_{pcbl} = 1 \quad (p \in \mathcal{P}^F), \\ x^L_{pck} &= \bar{x}^L_{pck}, \quad \sum_{c,k} x^L_{pck} = 1 \quad (p \in \mathcal{P}^L). \end{aligned} \tag{3}$$

A position containing a pair must be active, and candidate positions are used consecutively:

$$\begin{aligned} x^F_{pcbl} &\le u^F_{cb}, \quad u^F_{c,b+1} \le u^F_{cb}, \\ x^L_{pck} &\le u^L_{ck}, \quad u^L_{c,k+1} \le u^L_{ck}. \end{aligned} \tag{4}$$

For each candidate operation $o$, activation and duration satisfy

$$0 \le s_o \le M b_o, \qquad 0 \le e_o \le M b_o, \qquad e_o = s_o + d_o b_o. \tag{5}$$

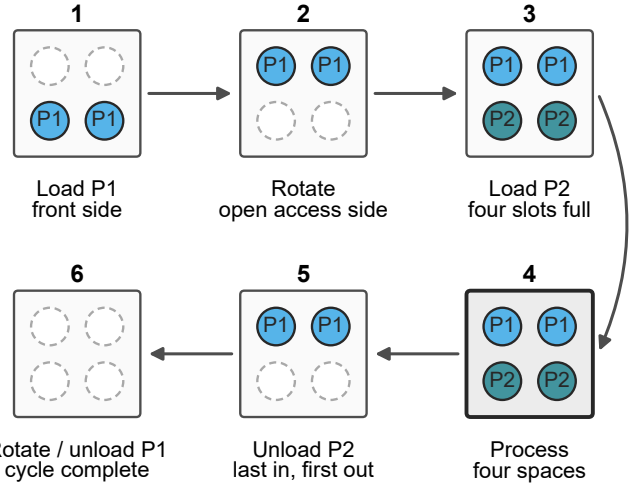



Fig. 3. $4 \times 1$ parallel processing. A regular all-product batch uses two product PW pairs, comprising four product wafers, and receives one synchronous full-occupancy process before last-in–first-out unloading; a dummy or mixed PW pair can complete the final short batch.

For every required process arc $(o, o') \in \mathcal{A}$,

$$b_{o'} \le b_o, \qquad s_{o'} \ge e_o + \delta_{oo'} - M(2 - b_o - b_{o'}). \tag{6}$$

### D. Occupancy and event order in the two processing modes

For $4 \times 1$, let $A^F_{cb} = \sum_{p,l} x^F_{pcbl}$ be the number of product pairs assigned to batch $(c, b)$. Full occupancy and dummy filling are

$$\begin{aligned} A^F_{cb} &= \sum_{p \in \mathcal{P}^F} \sum_{l \in \mathcal{L}} x^F_{pcbl}, \\ \sum_{p \in \mathcal{P}^F} x^F_{pcbl} + f^F_{cbl} &= u^F_{cb} \qquad (l \in \mathcal{L}), \\ \sum_{l \in \mathcal{L}} f^F_{cbl} &= 2u^F_{cb} - A^F_{cb}. \end{aligned} \tag{7}$$

Active batches are consecutive and dummy filling is restricted to the last active batch:

$$\begin{aligned} A^F_{cb} &\ge u^F_{cb}, & u^F_{c,b+1} &\le u^F_{cb}, \\ A^F_{cb} &\ge 2u^F_{c,b+1}, & t^F_{cb} &\le u^F_{cb}, \\ t^F_{cb} &\le 1 - u^F_{c,b+1}, & t^F_{cb} &\ge u^F_{cb} - u^F_{c,b+1}. \end{aligned} \tag{8}$$

Equation (8) requires at least one product pair in an active batch and two product pairs if a later batch is active. The last three inequalities set $t^F_{cb} = 1$ exactly at the active tail; the final candidate uses $t^F_{cB} = u^F_{cB}$.

Let $FL, BL, P, BU, FU$ denote front load, back load, processing, back unload, and front unload. Then

$$\begin{aligned} e^{F,\nu}_{cb} &= s^{F,\nu}_{cb} + d^F_\nu u^F_{cb} \\ &\quad \nu \in \{FL, BL, P, BU, FU\}, \\ s^{F,BL}_{cb} &\ge e^{F,FL}_{cb} + r_c - M(1 - u^F_{cb}), \\ s^{F,P}_{cb} &= e^{F,BL}_{cb}, \\ s^{F,BU}_{cb} &\ge e^{F,P}_{cb} - M(1 - u^F_{cb}), \\ s^{F,FU}_{cb} &\ge e^{F,BU}_{cb} + r_c - M(1 - u^F_{cb}). \end{aligned} \tag{9}$$

Equation (9) loads the front side, rotates to load the back, starts synchronous processing after the back load, and then unloads the back side before rotating and unloading the front. Consecutive batches are linked by $s^{F,FL}_{c,b+1} \ge e^{F,FU}_{cb} - M(1 - u^{F}_{c,b+1})$.

For $2 \times 2$, each product pair occupies one ordered chain position:

$$u^L_{ck} = \sum_{p \in \mathcal{P}^L} x^L_{pck}, \qquad u^L_{c,k+1} \le u^L_{ck}, \qquad a^L_c = u^L_{c1}. \tag{10}$$

Let $q^-_c(k)$ and $q^+_c(k)$ denote the first and second process cycles of chain position $k$. The pair–cycle incidence is

$$\begin{aligned} q^+_c(k) &= q^-_c(k) + 1, \\ \mu^L_{pcq} &= \sum_{\substack{k \in \mathcal{K}_c: \\ q \in \{q^-_c(k), q^+_c(k)\}}} x^L_{pck}, \\ \sum_{q \in \mathcal{Q}_c} \mu^L_{pcq} &= 2 \sum_{k \in \mathcal{K}_c} x^L_{pck}. \end{aligned} \tag{11}$$

Equation (11) maps every assigned $2 \times 2$ product-containing PW pair to exactly two adjacent process cycles. Every product member inherits the same $\mu^L_{pcq}$ incidence, so a product is not omitted even when an odd tail uses a P+D mixed PW pair. The equation also distinguishes the modes explicitly: a $2 \times 2$ product-containing pair spans two cycles, whereas two $4 \times 1$ side pairs are processed once in one full batch.

The generated layout supplies the number $\bar{Q}_c$ of process cycles, and $h^L_{ck}$ identifies the chain tail:

$$\begin{aligned} v^L_{cq} &= \begin{cases} a^L_c, & q \le \bar{Q}_c, \\ 0, & q > \bar{Q}_c, \end{cases} \\ \sum_k h^L_{ck} &= a^L_c, \qquad h^L_{ck} \le u^L_{ck}, \qquad h^L_{ck} + u^L_{c,k+1} \le 1, \\ h^L_{ck} &\ge u^L_{ck} - u^L_{c,k+1}. \end{aligned} \tag{12}$$

Equation (12) gives $n+1$ process cycles for a segment with $n$ product-containing PW pairs: the head is dummy PW pair+P1, internal cycles are P$i$+P$i+1$, and the tail is P$n$+dummy PW pair. The final relations make $h^L_{ck}$ one only at the last active product position.

For a legal chain segment $(q_0, \dots, q_T)$, let $H_0$, $B_q$, and $T_0$ denote head loading, internal exchange, and tail unloading. Then

$$\begin{aligned} e^L_{cq} - s^L_{cq} &= d^L v^L_{cq}, \\ s^L_{cq_0} &= e^{H_0}_c, \\ s^B_{cq} &\ge e^L_{c,q-1} - M(1 - a^L_c), \\ s^L_{cq} = e^B_{cq} \quad (q_0 < q \le q_T), & \\ s^{T_0}_c &\ge e^L_{cq_T} - M(1 - a^L_c). \end{aligned} \tag{13}$$

Equation (13) begins the first process immediately after head loading, performs an internal exchange only after the previous cycle, and unloads the tail only after the last process. It directly realizes dummy+P1, P1+P2, …, Pn+dummy.

### E. Cleaning and reusable dummy wafers

Cleaning segments are generated from $H$ and complete process boundaries and enter the MILP as fixed binary indicators:

$$z_{ce} = \bar{z}_{ce}, \qquad a^F_{cbe} = \bar{a}^F_{cbe} u^F_{cb}, \qquad a^L_{cqe} = \bar{a}^L_{cqe} v^L_{cq}. \tag{14}$$

Equation (14) prevents arbitrary cleaning locations: $4 \times 1$ may split only at batch boundaries and $2 \times 2$ only after a complete chain segment, while the MILP still schedules process and cleaning times.

Let $N_{ce}$ be the number of active process cycles in segment $e$:

$$\begin{aligned} N_{ce} &= \sum_b a^F_{cbe} + \sum_q a^L_{cqe}, \qquad z_{ce} \le N_{ce} \le H z_{ce}, \\ q^{\mathrm{cl}}_{ce} &= z_{c,e+1} + \tau_{ce}, \qquad z_{c,E_c+1} = 0. \end{aligned} \tag{15}$$

Equation (15) places at least one and at most $H$ process cycles in an active segment. It activates an internal cleaning before every subsequent production segment and, when the final segment exactly exhausts $H$ cycles, sets $\tau_{ce} = 1$ to activate terminal cleaning. Both are complete equipment actions; only the terminal action has no subsequent product processing.

Let $\mathcal{V}_c$ be the union of $4 \times 1$ batches and $2 \times 2$ cycles and let $a_{vce}$ take the corresponding $a^F$ or $a^L$. Cleaning order and the physical cleaning chain satisfy

$$\begin{aligned} s_v &\ge e^{\mathrm{cl}}_{c,e-1} - M(1 - a_{vce}) && (e > 1), \\ e_v &\le s^{\mathrm{cl}}_{ce} + M(1 - a_{vce}) + M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,BL}}_{ce} &\ge e^{\mathrm{cl,FL}}_{ce} + r_c - M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,P}}_{ce} &\ge e^{\mathrm{cl,BL}}_{ce} - M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,BU}}_{ce} &\ge e^{\mathrm{cl,P}}_{ce} - M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,FU}}_{ce} &\ge e^{\mathrm{cl,BU}}_{ce} + r_c - M(1 - q^{\mathrm{cl}}_{ce}). \end{aligned} \tag{16}$$

Equation (16) starts a segment after the preceding cleaning and finishes it before the next cleaning. A cleaning task loads both chamber sides with dummies, rotates, processes, and unloads in reverse, thereby occupying the VTR, chamber, and four physical dummies rather than adding virtual downtime.

Dummy demand $j$ is activated by a physical event:

$$q_j = \begin{cases} f^F_{cbl}, & j \text{ is a final-}4 \times 1 \text{ filling demand}, \\ a^L_c, & j \text{ is a chain endpoint demand}, \\ q^{\mathrm{cl}}_{ch}, & j \text{ is a cleaning demand}. \end{cases} \tag{17}$$

Equation (17) prevents dummies from becoming an unlimited abstract filler: a physical demand exists only for a final batch, chain head/tail, or active cleaning task.

Each active demand selects one physical dummy from inventory:

$$\sum_{\tau \in \mathcal{T}_c} g_{j\tau} = q_j, \qquad j \in \mathcal{J}^D_c. \tag{18}$$

### F. Robots, load locks, chambers, and time resources

ATR, AL, VTR, and each rotary chamber are unary resources; the upper and lower load locks each have two physical slots; each dummy in inventory is a reusable slot. An operation $o$ requiring $\kappa_o$ slots satisfies

$$\sum_{\ell \in \Gamma_o} h_{o\ell} = \kappa_o b_o. \tag{19}$$

If operations $o$ and $o'$ may compete for slot $\ell$ of resource $r$, then

$$\begin{aligned} s_{o'} &\geq e_o + \rho^r_{oo'} - M(1 - y^r_{oo'}) - M(1 - h_{o\ell}) - M(1 - h_{o'\ell}), \\ s_o &\geq e_{o'} + \rho^r_{o'o} - M y^r_{oo'} - M(1 - h_{o\ell}) - M(1 - h_{o'\ell}). \end{aligned} \tag{20}$$

Equation (20) makes $y^r_{oo'}$ choose one order whenever two actions use the same slot and includes empty robot travel or equipment reset through $\rho^r_{oo'}$. It applies to ATR, AL, both load locks, VTR, rotary chambers, and individual dummies, preventing concurrent robot actions and reuse before release.

An individual product or dummy stage must coincide with its selected batch, chain position, or cleaning window. For selection $x$ and reference stage $[s^{\mathrm{ref}}, e^{\mathrm{ref}}]$,

$$\begin{aligned} -M(1 - x) \leq s_o - s^{\mathrm{ref}} \leq M(1 - x), \\ -M(1 - x) \leq e_o - e^{\mathrm{ref}} \leq M(1 - x). \end{aligned} \tag{21}$$

If pair $p$ uses one paired transfer on robot route $a$, the two wafer stages are synchronized:

$$\begin{aligned} -M(1 - \gamma_{pa}) \leq s_{w_1 a} - s_{w_2 a} \leq M(1 - \gamma_{pa}), \\ -M(1 - \gamma_{pa}) \leq e_{w_1 a} - e_{w_2 a} \leq M(1 - \gamma_{pa}). \end{aligned} \tag{22}$$

For the entry and exit events of wafer $w$ in restricted equipment region $r$,

$$0 \leq s_{\mathrm{out}(w,r)} - e_{\mathrm{in}(w,r)} \leq \overline{R}_{wr}. \tag{23}$$

Equation (23) limits the physical stay in an upper load lock, chamber, lower load lock, or restricted transport region. Load-lock occupation also spans placement, pump/vent, opposite-side pickup, and pressure reset, so placement by a robot is not mistaken for slot release.

### G. Makespan-first two-stage optimization

Let $C_w$ be the time at which product wafer $w$ completes the lower-load-lock return and reaches its load port. Stage one solves

$$C_w = e_{w,\mathrm{return}}, \qquad C_{\max} \geq C_w \ \ (w \in \mathcal{W}), \qquad \min_{\xi \in \mathcal{F}} C_{\max}. \tag{24}$$

Equation (24) measures makespan at the last product wafer's return to LP after its complete end-to-end route. Internal cleaning affects $C_{\max}$ through resource occupancy and precedence that delay subsequent product returns. Terminal cleaning remains in the action sequence, but its completion time does not directly lower-bound $C_{\max}$. It can still affect other chambers' product returns through competition for the shared VTR; this indirect effect is determined by the complete schedule. WPH uses product-return makespan as its denominator. The feasible region $\mathcal{F}$ is defined by Eqs. (1)–(23) expanded over all applicable indices.

Given stage-one incumbent $\widehat{\xi}$ and makespan $\widehat{C}$, stage two fixes the discrete structure indexed by $\mathcal{D}_{\mathrm{fix}}$ and minimizes linear schedule quality within the makespan tolerance:

$$\begin{aligned} \xi_d = \widehat{\xi}_d \quad (d \in \mathcal{D}_{\mathrm{fix}}), \qquad C_{\max} \leq \widehat{C} + \varepsilon_C, \\ L_{\mathrm{sum}} = \sum_{\alpha \in Q^w} \omega_\alpha, \qquad L_{\max} \geq \omega_\alpha, \\ \Delta_{ceq} = (h_{ce,q+1} - h_{ceq}) - (h_{ceq} - h_{ce,q-1}), \\ v_{ceq} \geq \Delta_{ceq}, \qquad v_{ceq} \geq -\Delta_{ceq}, \\ \eta_\alpha \geq \omega_\alpha - \bar{\omega}_\alpha, \qquad \eta_\alpha \geq 0, \\ \min \Phi = \lambda_1 L_{\mathrm{sum}} + \lambda_2 L_{\max} + \lambda_3 \sum_{(c,e,q) \in \mathcal{I}_h} v_{ceq} + \lambda_4 \sum_{\alpha \in Q^w} \eta_\alpha. \end{aligned} \tag{25}$$

Equation (25) preserves the stage-one makespan bound and fixes the principal discrete routes and resource orders. Retained paired-transfer choices and continuous times can change to reduce total wait, maximum wait, and variation between consecutive processing intervals within each segment. Optional soft-limit excess can also be penalized. The experiments use $(\lambda_1, \lambda_2, \lambda_3) = (1, 10, 25)$ and $\varepsilon_C = 10^{-6}$. With the soft waiting limit disabled, no excess variables are generated. For a time-limited first stage, the best incumbent value $\widehat{C}$ supplies the second-stage bound; variables in $\mathcal{D}_{\mathrm{fix}}$ are fixed while designated paired-transfer variables remain adjustable.

## III. Structure-Aware Solution Framework

SCIP generates valid inequalities, checks incumbents and maintains primal/dual bounds [23], [24]. SPBS provides an initial feasible schedule and RDMCT-HBS selects from solver-generated cuts. Neither component changes the model's integer feasible set. The supplementary material gives the expanded network, feature and training definitions.

### A. SPBS initial solution

SPBS constructs a conservative discrete schedule from the recipe and tool layout. Products are paired in input order; parallel-mode pairs are placed sequentially and chained-mode pairs round-robin across chambers. Complete chains determine cleaning boundaries, and deterministic load-lock slots break symmetry. ATR/VTR order follows product progression and dummy return.

A restricted MILP fixes the binary skeleton and solves for compatible event times. Five relaxation levels successively release VTR order, load-lock order, other resource orders, and slot choices, retaining prescribed structural assignments. Large instances start at less restrictive levels. Each subproblem uses a zero objective, feasibility emphasis, aggressive heuristics and no cut separation, and stops at its first feasible solution. Failure to find one within a level's budget is not an infeasibility certificate.

After completing auxiliary schedule-quality variables, a candidate must pass the original-model feasibility check before becoming an incumbent. The main solve restores original bounds. These initial schedules are precomputed in the reported benchmark; their construction cost is excluded from main-solve timing and PDI. Thus the SPBS comparison estimates the value

of an available feasible start, not an initialization-inclusive speedup.

### B. Role-aware hierarchical cut selection

The 2023 HEM sequence model is the direct learned baseline [22]; its later sequence/set extension is related work [25]. The 13 baseline features measure objective parallelism, efficacy, support, integral support, violation, and four statistics each of cut and objective coefficients. Ten added features describe five variable-role coefficient masses, finite-bound mass, positive-coefficient mass, discrete–continuous coupling, dominant-role mass and role entropy. Roles are binary, general integer, objective-active continuous, other continuous auxiliary, and residual variables. For cut coefficients $a_{ij}$, role mass is

$$p_{ig} = \frac{\sum_{j:x_j \in g} |a_{ij}|}{\sum_j |a_{ij}| + \varepsilon}. \tag{26}$$

Feature extraction uses variable type, objective coefficient and global bounds, so renaming variables leaves the representation unchanged.

After layer normalization, separate linear projections of the 13 generic and ten role features are fused with a sigmoid gate. A one-layer 128-unit LSTM encodes candidates; a pointer decoder uses one glimpse and masks previously selected rows [26]. A high-level policy predicts the retained count. At inference, width-three beam search retains the highest accumulated log-probability prefixes [27]. Candidates are capped at 128 in efficacy order; at most 20% and 16 rows are retained, subject to SCIP's limit. Forced cuts remain under solver control.

The first $\lceil 0.25k \rceil$ decoded rows serve as fixed anchors. Greedy completion selects remaining rows from at most $2k$ policy/heuristic candidates by marginal gain of

$$F(S) = \sum_{i \in S} (0.35 q_i + 0.20 r_i) + 0.25 \sum_g d_g \sqrt{\sum_{i \in S} p_{ig}} + \frac{0.20}{|\mathcal{R}|} \sum_{u \in \mathcal{R}} \max_{i \in S} s_{ui}. \tag{27}$$

Here $q_i$ combines instantaneous cut quality and role descriptors, $r_i$ is a normalized pointer-rank prior, $d_g$ is pool role demand, and $s_{ui}$ is clipped cosine similarity. Nonnegative weights make this a monotone submodular surrogate. For fixed anchors and pool, greedy residual completion has the standard $1-1/e$ approximation property [28]; this is not a guarantee for PDI or solve time. Anchor order precedes greedy-completion order when cuts enter the LP.

### C. Training protocol

Training uses five instances, five independent training seeds and the fixed epoch-60 checkpoint. Each epoch collects eight 30-s/1024-MB rollouts with two workers. The terminal PDI is the cost; the first eligible candidate-set action supplies the high/low-level log-probability sample. Parallel hierarchical REINFORCE uses separate exponential-moving-average baselines (decay 0.9), sequence-length-normalized low-level log probability, and gradient clipping at 2 [29], [30]. The high level updates every two epochs. Adam learning rates start at $10^{-4}$ and $5\times10^{-4}$ and decay by 0.96 every ten epochs. HEM, feature-only and complete policies are trained separately. Stochastic training and beam inference are distinguished; test-instance performance is not used to select checkpoints.

## IV. Computational experiments and results

### A. Experimental design and metrics

The experiments examine four questions: how lot size and cleaning interval affect mode preference; whether equal-budget two-stage optimization improves waiting and cadence; how SPBS and structure-aware cut selection separately affect feasible-schedule availability and primal–dual convergence; and how scale, composition, and parameter changes affect computational performance. The evidence comprises mode comparisons, the direct HEM comparison, 960 paired SPBS runs, 1500 equal-budget refinement runs, 300 factorial runs, 80 one-factor perturbation runs, and 729 distribution-shift runs across all methods on 40–64-wafer lots. Resource Gantt charts show chained-mode operation for 16 wafers at $H = 10$ and $H = 3$. Every feasible schedule used in the analysis passes SCIP validation against the original model and independent event-time checks.

Within each comparison family, methods share the MILP, SCIP version, instances, random seeds, time and memory limits. The algorithm benchmark uses root-only cut separation, including its configured SCIP baseline. Let $P(t)$ denote the primal bound of the minimization problem at time $t$, namely the objective value of the best incumbent, and let $D(t)$ denote the dual bound, namely the global lower bound certified by the search tree. Define $\Delta(t) = |P(t) - D(t)|$, $m(t) = \min\{|P(t)|, |D(t)|\}$, and $M(t) = \max\{|P(t)|, |D(t)|\}$. When the two bounds are finite, have the same sign, and differ, the terminal relative primal–dual gap is defined in SCIP as

$$G_{\text{end}} = 100 \, \frac{\Delta(T)}{m(T)}. \tag{28}$$

SCIP sets $G_{\text{end}} = 0$ when the bounds coincide and returns $+\infty$ when they have opposite signs or valid bounds are not both available. Equation (28) normalizes by the smaller absolute bound and consequently does not cap $G_{\text{end}}$ at 100%. A value above 100% indicates that the absolute bound difference exceeds the smaller absolute bound.

SCIP uses a different bounded normalization for the primal–dual integral. Define the instantaneous integral gap as

$$g_{\text{PDI}}(t) = 100 \, \frac{\Delta(t)}{M(t)}, \qquad \text{PDI} = \int_0^T g_{\text{PDI}}(t) \, \mathrm{d}t. \tag{29}$$

Equation (29) applies when the bounds are finite, have the same sign, and differ; SCIP sets $g_{\text{PDI}}(t) = 0$ for coincident bounds and $g_{\text{PDI}}(t) = 100$ otherwise. The instantaneous integral gap is therefore bounded to $[0, 100]$; lower PDI means that the bound gap closes earlier. PDI is zero only if the gap is zero for almost the entire timed horizon, whereas $G_{\text{end}} = 0$ only states that optimality is proved at termination. Both quantities follow SCIP definitions and use its solution statistics [23], [24].

The manufacturing instance is the inferential unit: random repeats are first aggregated within an instance, after which

paired instance differences use nonparametric 95% bootstrap intervals, Wilcoxon signed-rank tests, and Holm correction within each prespecified analysis family. Time-limited runs remain in the summaries so that random seeds are not treated as independent manufacturing evidence.

Two experiment families use different prescribed configurations. The frozen algorithm benchmark uses nominal 180-s durations for both a parallel process and each chained exposure, a 500-s module-residency bound, and $H = 5$ in the nominal core suite. Its PDI and schedule-quality comparisons concern methods solving the same instances; they do not compare equal-dose process modes. The mode-selection study instead uses 180 s versus 90+90 s and a 10000-s module-residency bound. The numerical values in Table I apply to the latter family. The product-return comparison uses equal cumulative nominal processing time: 180 s for one $4 \times 1$ exposure and 90+90 s for two $2 \times 2$ exposures. Recipe qualification is a prescribed input condition for this scheduling comparison.

The two chambers share the VTR. Displayed schedules pass original-model and event-time checks.

### *B. Process-mode selection driven by cleaning position and lot size*

Steady-state analyses of multi-space CTs usually measure cycle time or throughput [3]. For finite batches, we define $C_{\max}$ as the time when the last product wafer returns to an LP and define equivalent batch output as $\mathrm{WPH} = 3600|\mathcal{W}|/C_{\max}$. Pure $4 \times 1$ uses 180 s per wafer, pure $2 \times 2$ uses 90+90 s, and a 1:1 mixture assigns each product its mode-specific recipe. The cleaning interval $H$ counts full-occupancy cycles per chamber: a $4\times1$ batch counts once and each $2\times2$ exposure counts separately. Processing segments follow cycle limits and complete-chain boundaries, including 500-s cleaning and associated dummy loading, unloading, rotation, and transport.

Table II gives the $H = 3$–10 comparison. The $H = 4$–9 experiment contains 36 configurations with training and solver seeds fixed at 1, a 14400-s main-solve limit, and stage 2 disabled. All 36 schedules pass independent validation; 35 are proven optimal. For 16 wafers at $H = 5$, pure $2 \times 2$ reaches a feasible 907 s within that budget. A separate 28800-s solve of the same instance, initialized from this schedule, improves the upper bound to 898 s, with lower bound 694 s and terminal gap 29.39%. The improved schedule passes main-model and 4-s exchange checks. This extended-budget experiment determines the feasible upper bound used for mode comparison; its computation time is excluded from the fixed-budget algorithm comparison. With cleaning inactive, the 16-wafer pure $2 \times 2$ models at $H = 6$–10 are identical and have optimum 898 s.

For 8 wafers, chained processing is preferred throughout $H = 3$–10. For 16 wafers, parallel, mixed and chained modes are preferred at $H = 3$, $H = 4$, and $H = 5$–10, respectively. At $H = 5$, the validated 898-s chained schedule beats both competing optima even though its own optimality is unresolved. At $H = 6$–10, 898 s is proven optimal, 4.87% below parallel operation.

The 16-wafer chained layout contains four product pairs per chamber. It splits them as 2+2 at $H = 3$, 3+1 at $H = 4$, and one chain at $H \geq 5$. Equal internal-cleaning counts at $H = 3$ and 4 therefore need not give equal makespans. The study compares the joint effect of the cleaning limit and regenerated admissible chain layout.

The 24-wafer extension covers all three modes at $H = 3$–6 with 7200 s per configuration. Pure $4 \times 1$ has optimum 1268 s at each interval. At $H = 4$, pure $2 \times 2$ and mixed-mode lower bounds are 1342 and 1774 s; at $H = 5$, the pure $2 \times 2$ lower bound is 1440 s and the mixed optimum is 1796 s. These bounds establish pure $4\times1$ as preferred at both intervals. At $H = 6$, the mixed-mode upper bound of 1263 s is below the pure $4\times1$ optimum and pure $2\times2$ lower bound of 1440 s, establishing mixed-mode dominance; its terminal gap is 10.40%. At $H = 3$, a 28800-s solve for pure $2\times2$ obtains a validated 2220-s schedule, lower bound 2082 s, and gap 6.63%. This lower bound and the mixed-mode lower bound of 2302 s exceed the pure $4 \times 1$ optimum. The extended-budget solve determines mode ordering; its computation time is excluded from the fixed-budget algorithm comparison. Hence pure $4\times1$ is preferred for 24 wafers at $H = 3$–5 and mixed operation at $H = 6$. The difference from the 16-wafer $H = 5$ result demonstrates the dependence on lot size and complete-chain layout.

All 12 configurations in the 24-wafer $H = 7$–10 extension yield validated schedules. An independent solve for mixed operation at $H = 10$ proves a makespan of 1250 s within the 28800-s budget. The mixed-mode MILPs at $H = 7$–10 have identical objectives, constraints, and variable bounds, so all four intervals have the same optimal makespan. Pure $2\times2$ yields a feasible 1237-s schedule at $H = 8$–10. Transferring its production-event timing to the $H = 7$ model and scheduling two terminal cleanings also gives a 1237-s schedule that passes validation against the original model. The valid pure $2\times2$ intervals are $[712, 1237]$ s at $H = 7$ and $[842, 1237]$ s at $H = 8$–10. These upper bounds are below the mixed-mode optimum of 1250 s and pure $4 \times 1$ optimum of 1268 s. Thus, pure $2 \times 2$ is preferred throughout $H = 7$–10, with the constructed schedules reducing makespan by 1.04% and 2.44%, respectively. Extended-solve and schedule-transfer costs are accounted for separately from the fixed-budget algorithm comparison.

The resource Gantt charts show 16-wafer chained operation at $H = 10$ and 3. To study waiting at minimum makespan, the $H = 10$ pure $2\times2$ schedule is further optimized for total route wait under $C_{\max} \leq 898 + 10^{-6}$ s, with resource order allowed to change. The minimum route wait is 492 s, accumulated over six consecutive-operation intervals for each product wafer: 448 s from AL pair coordination, 36 s before post-processing unloading, and 8 s before return from the lower load lock. This route measure is distinct from the interval between consecutive in-chamber exposures. Each time axis ends at the last product return, $C_{\max}$. Every pure $4 \times 1$ process lasts 180 s and every pure $2 \times 2$ exposure lasts 90 s.

Chamber lanes distinguish gray exchange actions from pale-pink waiting. A combined exchange by the four-finger VTR takes 4 s. A split exchange includes 4-s unloading, at least 4 s of empty repositioning, and 4-s loading; other material actions may be coordinated between unloading and loading. In Fig. 4, the first CH2 inter-exposure interval is 63 s: 59 s waiting

TABLE I

EXPERIMENTAL PARAMETERS FOR THE CLEANING-FREQUENCY MODE EXPERIMENT AND REPRESENTATIVE GANTT CHARTS UNDER EQUAL TOTAL NOMINAL PROCESSING TIME.

| Group | Parameter and value | Group | Parameter and value |
|---|---|---|---|
| Rotating chambers | CH2 and CH3; two shared rotating PMs with four spaces each | Process time | 180 s for $4 \times 1$; 90+90 s for $2 \times 2$ |
| Cleaning | At most $H$ = 3–10 effective process cycles between adjacent cleanings; pure-dummy cleaning 500 s | Chamber motion | 180° rotation 2 s; inter-action gap 1 s |
| ATR | One dual-hand robot, capacity 2; load/unload 4 s, complete loaded transfer 11 s | VTR | One four-hand robot, capacity 4; loaded transfer 4 s, empty reposition 4 s |
| ATR details | LL–LP empty return 6 s; PW-pair exchange at AL 8 s | AL | Capacity 1; minimum alignment occupancy 20 s per wafer |
| Load locks | LLupper pump/reset 30 s per slot; LLlower vent/reset 25 s | LL capacity | Two independent slots in each of upper and lower LLs |
| Material source | LP1–LP4, capacity 25 wafers each | Dummy wafers | Eight reusable wafers, four per chamber; storage capacity 10 |
| Batch and model | Product batch size 4; 13 modeled process steps | Chamber bound | At most 15 $4 \times 1$ batch slots per PM |
| Model bounds | Big-$M$ = 10000; module and robot residency bounds 10000 s | Waiting option | Avoidable-wait hard cap 0, meaning that this optional cap is disabled |
| Objective | Product-return makespan and waiting refinement | Policy | Fixed training seed 1, epoch 60, 23 features |
| Decoder and cuts | Beam width 3; cut pool 128, at most 16 cuts selected per round | Solver | SCIP seed 1; Available, validated initial schedules |
| Solve and selection | Product-return objective; makespan bounds for mode comparison; waiting-refined schedules for Gantt charts | Schedule validation | Original-model and independent validation for every schedule |

TABLE II

MAKESPANS, CLEANING COUNTS, AND PREFERRED MODES UNDER THE PRESCRIBED PW LAYOUTS FOR 8-, 16-, AND 24-WAFER LOTS AT $H$ = 3–10. TIMES ARE IN SECONDS.

| Lot (wafers) | $H$ | $C_{\max}$ (s) | | | $\Delta C_{\max}$ (s) | Total cleans | | | Preferred mode |
|---|---|---|---|---|---|---|---|---|---|
| | | Pure $4 \times 1$ | Pure $2 \times 2$ | 1:1 mixed | | Pure $4 \times 1$ | Pure $2 \times 2$ | 1:1 mixed | |
| 8 | 3 | 610 | 578 | 654 | -32 | 0 | 2 | 1 | Pure $2 \times 2$ |
| 8 | 4–10 | 610 | 578 | 654 | -32 | 0 | 0 | 0 | Pure $2 \times 2$ |
| 16 | 3 | 944 | 1402 | 1530 | 458 | 0 | 4 | 2 | Pure $4 \times 1$ |
| 16 | 4 | 944 | 1470 | 939 | 526 | 0 | 2 | 2 | 1:1 mixed |
| 16 | 5 | 944 | 898† | 939 | -46 | 0 | 2 | 0 | Pure $2 \times 2$ |
| 16 | 6–10 | 944 | 898 | 939 | -46 | 0 | 0 | 0 | Pure $2 \times 2$ |
| 24 | 3 | 1268 | 2220† | 2324† | 952 | 2 | 6 | 4 | Pure $4 \times 1$ |
| 24 | 4 | 1268 | 2928† | 1827† | 1660 | 0 | 4 | 2 | Pure $4 \times 1$ |
| 24 | 5 | 1268 | 1769† | 1796 | 501 | 0 | 2 | 2 | Pure $4 \times 1$ |
| 24 | 6 | 1268 | 1880† | 1263† | 612 | 0 | 2 | 1 | 1:1 mixed |
| 24 | 7 | 1268 | 1237† | 1250 | -31 | 0 | 2 | 0 | Pure $2 \times 2$ |
| 24 | 8–10 | 1268 | 1237† | 1250 | -31 | 0 | 0 | 0 | Pure $2 \times 2$ |

Cleaning counts sum internal and terminal cleans across both chambers. † denotes a feasible upper bound; other times are proven optima. $\Delta C_{\max}$ subtracts the listed pure $4 \times 1$ time from the pure $2 \times 2$ time. A preferred mode is established by comparing optima or by a feasible upper bound below the competing lower bounds.

followed by a 4-s combined exchange. The next product PW, W5/W6, enters the upper load lock at 299 s and completes vacuum-side preparation at 329 s. CH2 finishes processing at 270 s, waits until 329 s, and exchanges at 329–333 s. Most adjacent cycles in the $H = 3$ short-chain schedule use continuous 4-s combined exchanges; CH2 cycles five and six are separated by waits at 1106–1108 s and 1112–1116 s, unloading at 1108–1112 s, and loading at 1116–1120 s. The final exposure starts immediately at 1120 s and ends at 1210 s. Every exposure remains 90 s. Constraining all eight inter-exposure intervals in the same $H = 10$ model to exactly 4 s yields an independently validated optimum of 960 s, 62 s (6.90%) above 898 s. Continuous exchanges compress local intervals but restrict supply and return order on shared resources; allowing local waiting shortens this finite batch's makespan.

### C. Equal-budget refinement of waiting and cadence

Waiting-time constraints can propagate to manufacturing-system output and control performance [31]. For the same 600-s total budget, the complete two-stage workflow assigns 540 s to the main solve and up to 60 s to refinement, whereas the one-stage control assigns all 600 s to the main solve. The measured instance means show that the two-stage workflow reduces total route wait, maximum wait, cadence coefficient of variation, and cadence deviation by 51.91%,

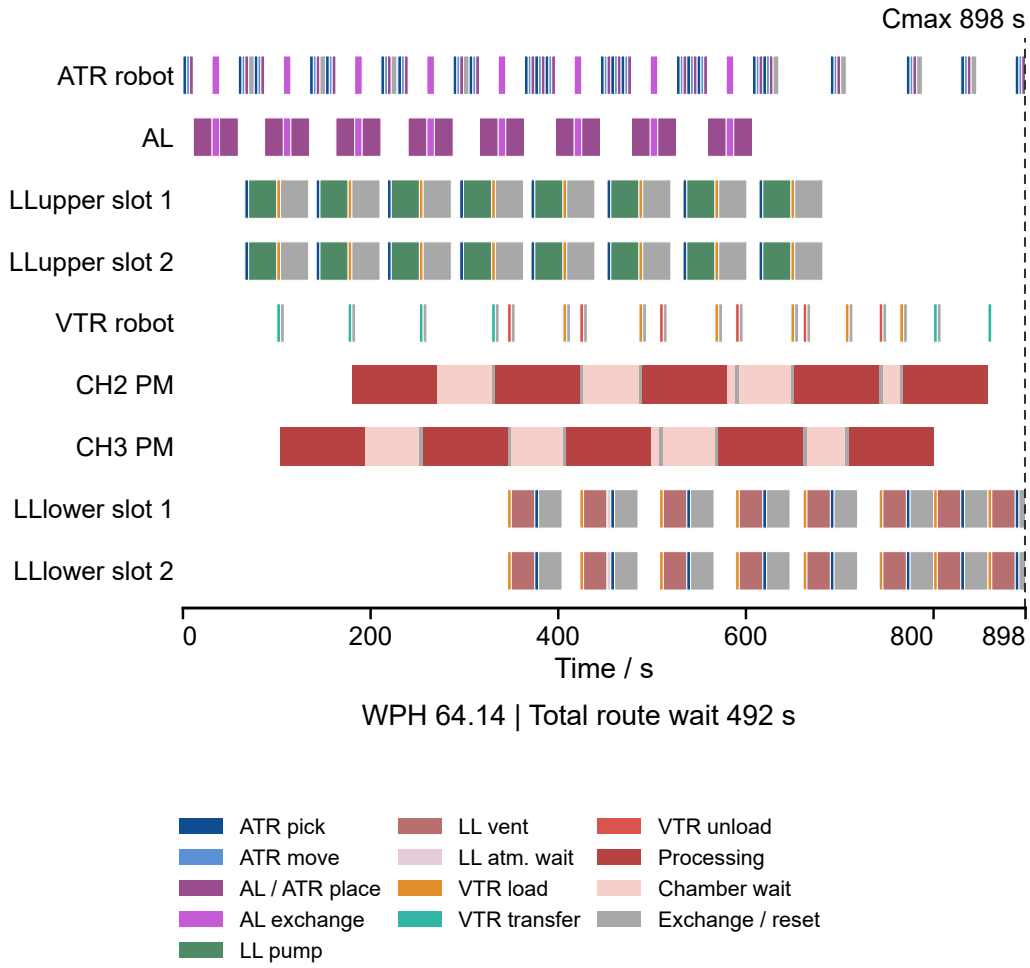



Fig. 4. 16-wafer pure $2 \times 2$ schedule at $H = 10$, without cleaning: $C_{\max} = 898$ s, WPH = 64.14 wafers/h. Refinement minimizes total route wait to 492 s at this makespan. Gray denotes 4-s chamber exchanges; pale pink denotes waiting.

68.89%, 39.70%, and 94.05%, respectively. All four paired instance differences remain significant after Holm correction, demonstrating that the two-stage workflow markedly improves waiting and cadence quality under the same total budget.

The quadratic-objective control and the complete linear refinement both use the 540+up-to-60-s split. Linear refinement yields a cadence deviation of 22.04, 63.38% below the quadratic profile's 60.18, and the matched difference remains significant after Holm correction ($p = 0.0377$); total route wait, maximum wait, and cadence coefficient of variation are similar overall. The linear objective therefore reduces the measured cadence deviation under the same budget.

Table III isolates the second stage and refinement objective, complementing the initialization ablation in Table IV.

### *D. SPBS and structure-aware cut selection*

Table IV summarizes the paired SPBS study with 48 instances and ten paired seeds per instance. The timed comparison measures main-solver performance after offline SPBS construction. Relative to the no-start condition, precomputed SPBS increases incumbent availability from 51.67% to 100%, a gain of 48.33 percentage points (95% interval $[34.58, 62.29]$). Mean PDI falls from 27473.05 to 20278.78, a decrease of 7194.27 or 26.19% (95% interval $[5145.78, 9317.61]$). Both paired differences remain significant after Holm correction ($p = 2.55 \times 10^{-5}$ and $p = 3.46 \times 10^{-8}$). Precomputed SPBS therefore improves feasible-schedule availability and primal–dual convergence during the subsequent solve.

HEM-13D is the most direct learning baseline for the structure-aware cut selector. Both methods share SPBS, training instances, random seeds, candidate limits, hidden dimensions, and a 600-s solve budget, with 500 runs per method. Table V includes this prespecified direct comparison. Relative to HEM-13D, RDMCT-HBS changes mean PDI from 16823.18 to 16585.70, mean solve time from 219.65 to 218.32 s, mean terminal gap from 127.24% to 126.01%, and proven-optimal rate from 62.4% to 63.2%. All 500 complete-method runs produce feasible schedules. After aggregating repeats over 20 manufacturing instances, all four paired mean-difference 95% bootstrap intervals include zero and all Holm-adjusted $p$-values are 1.0. The observed means are close under the com-

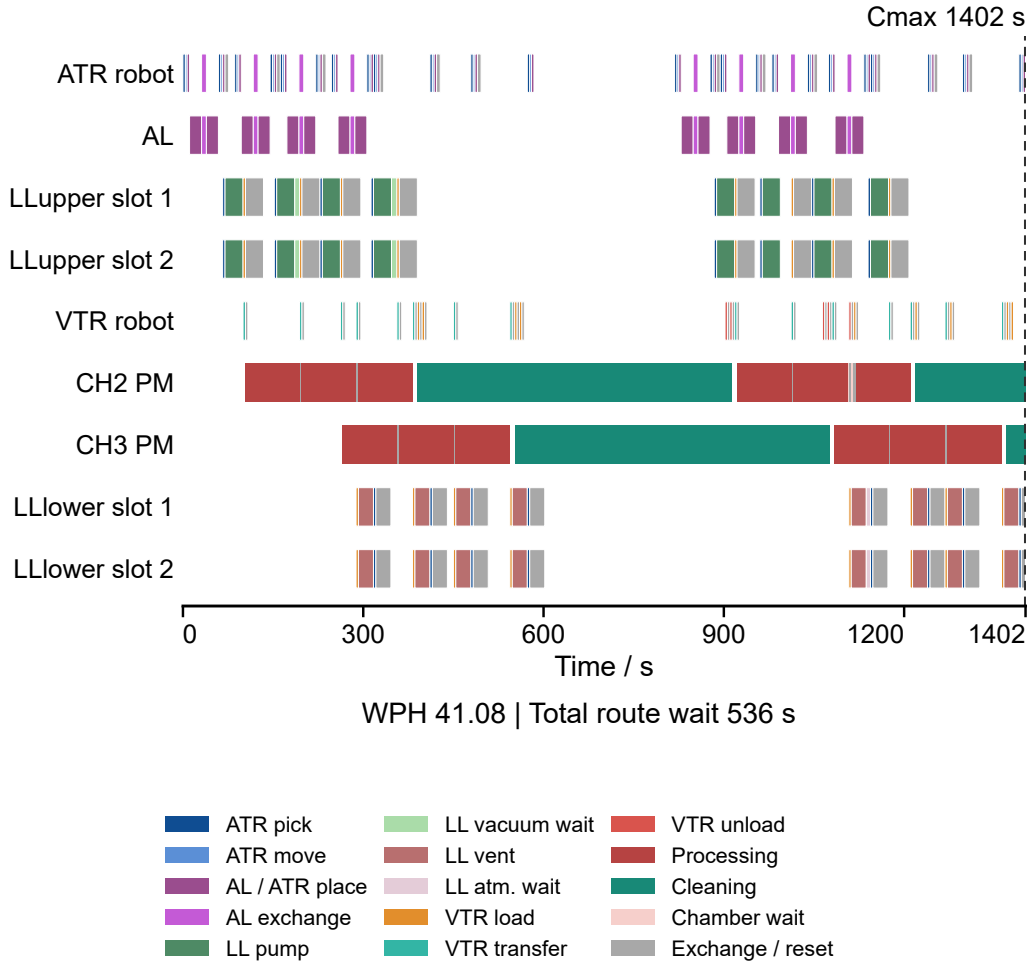



Fig. 5. 16-wafer pure $2 \times 2$ schedule at $H = 3$: two internal and two terminal cleans, $C_{\max}$ = 1402 s, WPH = 41.08 wafers/h, and total route wait = 536 s. The horizon ends at product return; exchanges and waiting are shown separately.

TABLE III
ABLATION OF SECOND-STAGE REFINEMENT AND OBJECTIVE FORM. EACH CONFIGURATION CONTAINS 20 INSTANCES AND 500 RUNS UNDER A 600-S TOTAL BUDGET. ENTRIES ARE INSTANCE-AGGREGATED MEANS; LOWER IS BETTER.

| Configuration | Total wait (s) | Max. wait (s) | Cadence CV | Deviation (s) |
|---|---|---|---|---|
| Full linear refinement | 2596.01 | 296.33 | 0.130 | 22.04 |
| Quadratic refinement | 2469.47 | 293.27 | 0.134 | 60.18 |
| Without stage two | 5398.60 | 952.57 | 0.216 | 370.41 |

TABLE IV
EFFECT OF PRECOMPUTED SPBS ON TIME-LIMITED MILP SEARCH; CONSTRUCTION TIME IS EXCLUDED. EACH CONDITION CONTAINS 48 INSTANCES AND TEN PAIRED SEEDS PER INSTANCE, TOTALING 480 RUNS.

| Metric | No start | SPBS | Observed change |
|---|---|---|---|
| Incumbent rate (%) | 51.67 | 100.00 | +48.33 points |
| Mean PDI | 27473.05 | 20278.78 | -26.19% |

mon SPBS condition; the paired tests provide the statistical context for these changes.

Table V adds default SCIP, validation-tuned adaptive cut scoring, and component ablations from the same frozen experiment. Nonlearning methods use five solver seeds per instance (100 runs each); each learned method additionally uses five training seeds (500 runs each). Configured SCIP has the highest observed optimal rate and lowest mean terminal gap, while the complete method has the lowest observed PDI and solve time. There is no consistent dominance across metrics.

The comparison jointly evaluates role-aware representation, sequence decoding, and structural completion under common initialization and solver budgets, quantifying their effects on PDI, solution time, terminal gap, and proven-optimal rate with shared SPBS.

TABLE V
COMPLETE FROZEN 2700-RUN COMPARISON WITH COMMON INITIAL SCHEDULES AND 600-S SOLVER BUDGETS. VALUES ARE MEANS OVER 20 INSTANCE AGGREGATES. GAP USES SCIP'S TERMINAL DEFINITION.

| Method | Runs | PDI | Solve time (s) | Gap (%) | Optimal (%) |
|---|---|---|---|---|---|
| Configured SCIP | 100 | 16919.01 | 226.05 | 119.54 | 64.0 |
| Adaptive cut scoring | 100 | 16854.87 | 222.34 | 127.18 | 62.0 |
| HEM-13D | 500 | 16823.18 | 219.65 | 127.24 | 62.4 |
| 23D features only | 500 | 16620.91 | 219.42 | 125.33 | 62.6 |
| HEM + beam | 500 | 16829.59 | 219.95 | 127.19 | 62.4 |
| Structure + greedy | 500 | 16598.11 | 218.44 | 126.03 | 63.2 |
| RDMCT-HBS | 500 | 16585.70 | 218.32 | 126.01 | 63.2 |

### *E. Scale, recipe composition, and computational difficulty*

The factorial design contains $4 \times 5 \times 3 = 60$ wafer-count–recipe-composition–process-scale cells with five solver seeds per cell, for 300 runs. The results show a sharp transition. All 75 runs at both 8 and 16 wafers prove optimality, with mean PDI values of 28.13 and 1876.54 and mean solution times of 0.34 and 33.86 s. At 24 and 32 wafers, the proven-optimal rates fall to 36% and 20%, mean PDI rises to 31155.94 and 32959.89, and mean solution time reaches 408.55 and 433.20 s.

The prespecified PDI factorial model has $R^2 = 0.998$. Under the 1:1 mixed recipe, the additional interaction coefficients at 24 and 32 wafers are 37729.48 and 40788.63, while the 95% intervals for the process-scale main effect and all prespecified process-scale interactions include zero. Over the tested 0.9–1.1 process-scale range, computational difficulty is driven primarily by the interaction between batch size and mixed-flow composition, with a comparatively small process-scale effect.

### *F. Operation under parameter perturbations*

Prior CT studies show that cleaning actions and their triggering conditions alter robot-task sequences, cycle time, and wafer delay [13], [15]. The one-factor sensitivity study therefore covers eight prescribed perturbations of transport, processing, cleaning interval, and vacuum-domain dummy inventory, for 80 runs. Every run returns a solver-checked feasible schedule and 20 runs prove optimality; scenario mean PDI ranges from 28899.41 to 42478.32. A longer cleaning interval gives the lowest mean PDI and slower transport the highest, describing the variation in search performance across the tested parameter settings.

### *G. Feasible-schedule delivery under distribution shift*

The distribution-shift suite enlarges batches to 40–64 wafers and changes mixed-flow composition, producing 729 runs across all compared methods over nine shifted manufacturing scenarios, including 135 runs of the complete method. This experiment evaluates schedule availability and search performance under changes in scale and composition. All 729 runs return an incumbent that passes SCIP and independent post-solve checks. The shared SPBS–MILP–SCIP chain therefore continues to output a validated resource-action schedule over the tested changes in scale and recipe composition. The proven-optimal rate is 0%. The complete method has mean PDI 73919.15, compared with 73670.99 for default SCIP and 70915.54 for adaptive cut scoring. Feasible-schedule availability is 100% across this suite, while adaptive cut scoring has the lowest observed mean PDI.

### *H. Discussion of mode selection and computational performance*

Mode preference depends on where cleaning occurs within a finite batch. Internal cleaning delays subsequent chains, whereas terminal cleaning affects the product-return objective differently. Mixed operation is preferred at $H = 4$ for 16 wafers and at $H = 6$ for 24 wafers. Thus, single-cycle processing capability or cleaning interval alone does not determine the preferred finite-batch mode; lot size and complete-chain layout also matter.

Mode preference can be established directly by comparing a feasible upper bound with competing lower bounds. For 24 wafers at $H = 7$–$10$, the pure $2 \times 2$ upper bound of 1237 s lies below both competing optima, establishing its preference among the compared layouts. Mode comparisons determine process choices for a given lot size and cleaning interval, while equal-budget algorithm comparisons quantify search performance.

The solver components address different schedule-performance measures. Precomputed SPBS improves feasible-schedule availability in subsequent time-limited search, and two-stage refinement improves waiting and cadence. With common SPBS, the complete cut selector and HEM have close observed mean metrics. Scale-extension experiments achieve 100% feasible-schedule availability, and adaptive scoring yields the lowest observed mean PDI. These results quantify schedule availability, timing quality, and bound convergence separately. The parameter-perturbation study reoptimizes deterministic instances and does not test execution under random disturbances. A feasible returned schedule satisfies the modeled constraints; this does not prove closed-loop stability, recovery from breakdowns, or a hard online response-time guarantee.

## V. CONCLUSIONS

The proposed two-stage MILP coordinates product and reusable dummy flows, chamber occupancy, cleaning and shared-resource timing for finite batches. Under prescribed qualified recipes and pair layouts, the preferred mode depends on lot size and cleaning-induced chain segmentation. Chained processing reaches 898 s for a 16-wafer lot without activated

cleaning, compared with 944 s for parallel processing. Valid upper and lower bounds establish the reported mode choices without treating a time limit as an infeasibility proof.

The frozen computational evidence supports the scheduling value of precomputed feasible starts and equal-budget waiting/cadence refinement. It does not establish statistical superiority of the complete cut selector over HEM, nor a universal out-of-distribution advantage. Further work should measure initialization-inclusive decision latency, relax prescribed pair placement, and validate disturbance handling and recipe qualification on physical equipment.

## Acknowledgment

OpenAI ChatGPT and Codex assisted with language revision and Chinese–English translation throughout the manuscript, the restructuring of the Abstract, Note to Practitioners, Introduction and Discussion, LaTeX preparation, and development/debugging of the numerical solution and visualization code described in Sections III–IV. Reported numerical results are retained from solver experiments; no new experimental outcomes were generated for this manuscript conversion.

## References

[1] C. R. Pan, M. C. Zhou, Y. Qiao, and N. Q. Wu, "Scheduling cluster tools in semiconductor manufacturing: Recent advances and challenges," *IEEE Transactions on Automation Science and Engineering*, vol. 15, no. 2, pp. 586–601, 2018.

[2] L. Gu, N. Wu, Y. Qiao, S. Zhang, and T. Li, "Scheduling cluster tools with multi-space process modules and a multi-finger-arm robot in wafer fabrication subject to wafer residency time constraints," *Applied Sciences*, vol. 14, p. 9490, 2024.

[3] L. Gu, N. Wu, Y. Qiao, and S. Zhang, "Scheduling of Multi-finger-robotic Cluster Tools with Multi-space Process Modules," in *2024 International Conference on Networking, Sensing and Control (ICNSC)*. IEEE, 2024.

[4] T. L. Perkinson, P. K. McLarty, R. S. Gyurcsik, and R. K. Cavin, "Single-wafer cluster tool performance: An analysis of throughput," *IEEE Transactions on Semiconductor Manufacturing*, vol. 7, pp. 369–373, 1994.

[5] S. Rostami and B. Hamidzadeh, "Optimal scheduling techniques for cluster tools with process-module and transport-module residency constraints," *IEEE Transactions on Semiconductor Manufacturing*, vol. 15, pp. 341–349, 2002.

[6] N. Q. Wu and M. C. Zhou, "A closed-form solution for schedulability and optimal scheduling of dual-arm cluster tools with wafer residency time constraint based on steady schedule analysis," *IEEE Transactions on Automation Science and Engineering*, vol. 7, pp. 303–315, 2010.

[7] Y. Qiao, N. Q. Wu, and M. C. Zhou, "Petri net modeling and wafer sojourn time analysis of single-arm cluster tools with residency time constraints and activity time variation," *IEEE Transactions on Semiconductor Manufacturing*, vol. 25, pp. 432–446, 2012.

[8] Y. Qiao, N. Wu, and M. C. Zhou, "Scheduling of dual-arm cluster tools with wafer revisiting and residency time constraints," *IEEE Transactions on Industrial Informatics*, vol. 10, pp. 286–300, 2014.

[9] J. Wang, H. Hu, C. Pan, Y. Zhou, and L. Li, "Scheduling dual-arm cluster tools with multiple wafer types and residency time constraints," *IEEE/CAA Journal of Automatica Sinica*, vol. 7, pp. 776–789, 2020.

[10] J. Wang, C. Liu, M. C. Zhou, and A. Abusorrah, "Virtual cell-based scheduling approach to single-robotic-arm cluster tools subject to wafer residency time constraints," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 240–251, 2025.

[11] F. Yang, C. Li, L. Zhen, C. Zhang, and N. Wu, "Scheduling residency time-constrained single-armed cluster tools with two-wafer types via mixed integer programming model," *Expert Systems with Applications*, vol. 288, p. 128209, 2025.

[12] H. Kim, H. J. Kim, J. H. Lee, and T. E. Lee, "Scheduling dual-armed cluster tools with cleaning processes," *International Journal of Production Research*, vol. 51, pp. 3671–3687, 2013.

[13] T. S. Yu and T. E. Lee, "Wafer delay analysis and control of dual-armed cluster tools with chamber cleaning operations," *International Journal of Production Research*, vol. 58, pp. 434–447, 2020.

[14] Y. Qiao, Y. Lu, J. Li, S. Zhang, N. Wu, and B. Liu, "An efficient binary integer programming model for residency time-constrained cluster tools with chamber cleaning requirements," *IEEE Transactions on Automation Science and Engineering*, vol. 19, pp. 1757–1771, 2022.

[15] C. Li, F. Yang, and L. Zhen, "Efficient scheduling approaches to time-constrained single-armed cluster tools with condition-based chamber cleaning operations," *International Journal of Production Research*, vol. 60, pp. 3555–3568, 2022.

[16] Q. Zhu, J. Yuan, G. Wang, and Y. Hou, "Scheduling a single-arm two-cluster tool with a process module failure subject to wafer residency time constraints," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 667–679, 2025.

[17] D. Kim and H. J. Kim, "Noncyclic scheduling of cluster tools using deep reinforcement learning," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 24 009–24 023, 2025.

[18] H. J. Kim and J. H. Lee, "Scheduling cluster tools for concurrent processing: Deep reinforcement learning with adaptive search," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 3783–3796, 2025.

[19] Y. Tang, S. Agrawal, and Y. Faenza, "Reinforcement learning for integer programming: Learning to cut," in *Proceedings of the 37th International Conference on Machine Learning*. PMLR, 2020, pp. 9367–9376. [Online]. Available: https://proceedings.mlr.press/v119/tang20a.html

[20] Z. Huang, K. Wang, F. Liu, H. L. Zhen, W. Zhang, M. Yuan, J. Hao, Y. Yu, and J. Wang, "Learning to select cuts for efficient mixed-integer programming," *Pattern Recognition*, vol. 123, p. 108353, 2022.

[21] M. B. Paulus, G. Zarpellon, A. Krause, L. Charlin, and C. Maddison, "Learning to cut by looking ahead: Cutting plane selection via imitation learning," in *Proceedings of the 39th International Conference on Machine Learning*. PMLR, 2022, pp. 17 584–17 600. [Online]. Available: https://proceedings.mlr.press/v162/paulus22a.html

[22] Z. Wang, X. Li, J. Wang, Y. Kuang, M. Yuan, J. Zeng, Y. Zhang, and F. Wu, "Learning cut selection for mixed-integer linear programming via hierarchical sequence model," in *Proceedings of the Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=Zob4P9bRNcK

[23] T. Achterberg, "SCIP: Solving constraint integer programs," *Mathematical Programming Computation*, vol. 1, pp. 1–41, 2009.

[24] K. Bestuzheva, M. Besançon, W. K. Chen, A. Chmiela, T. Donkiewicz, J. van Doornmalen, L. Eifler, O. Gaul, G. Gamrath, A. Gleixner, L. Gottwald, C. Graczyk, K. Halbig, A. Hoen, C. Hojny, R. van der Hulst, T. Koch, M. E. Lübbecke, S. J. Maher, F. Matter, E. Mühmer, B. Müller, M. E. Pfetsch, F. Schlösser, F. Serrano, Y. Shinano, B. Sofranac, M. Turner, S. Vigerske, F. Wegscheider, P. Wellner, D. Weninger, and J. Witzig, "Enabling research through the SCIP optimization suite 8.0," *ACM Transactions on Mathematical Software*, vol. 49, pp. 22:1–22:21, 2023.

[25] J. Wang, Z. Wang, X. Li, Y. Kuang, Z. Shi, F. Zhu, M. Yuan, J. Zeng, Y. Zhang, and F. Wu, "Learning to cut via hierarchical sequence/set model for efficient mixed-integer programming," *arXiv preprint arXiv:2404.12638*, 2024. [Online]. Available: https://arxiv.org/abs/2404.12638

[26] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Advances in Neural Information Processing Systems*, 2015, pp. 2692–2700.

[27] J. Choo, Y. D. Kwon, J. Kim, J. Jae, A. Hottung, K. Tierney, and Y. Gwon, "Simulation-guided beam search for neural combinatorial optimization," in *Advances in Neural Information Processing Systems*, 2022.

[28] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions—i," *Mathematical Programming*, vol. 14, pp. 265–294, 1978.

[29] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, pp. 229–256, 1992.

[30] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proceedings of the 33rd International Conference on Machine Learning*. PMLR, 2016, pp. 1928–1937. [Online]. Available: https://proceedings.mlr.press/v48/mniha16.html

[31] J. H. Lee, C. Zhao, J. Li, and C. T. Papadopoulos, "Analysis, design, and control of Bernoulli production lines with waiting time constraints," *Journal of Manufacturing Systems*, vol. 46, pp. 208–220, 2018.